modal.com

Speculation Is All You Need | Modal Blog

22~27분

We are all-in on speculative decoding, and we'd like to tell you why.

But first: we're big fans of Z Lab's DFlash draft model architecture. That's why we released a state-of-the-art DFlash speculator for Qwen 3.5 397B-A17B this week and worked closely with SGLang to make sure its performance was world-beating.

It's also why we worked with Z Lab to train state-of-the-art speculators for more models in the Qwen series, which we're releasing on Hugging Face today:

- Qwen 3.6 35B-A3B-DFlash
- Qwen 3.5 4B-DFlash
- Qwen 3.5 9B-DFlash
- Qwen 3.5 27B-DFlash
- Qwen 3.5 35B-A3B-DFlash
- Qwen 3.5 122B-A10B-DFlash

On top of the strong baseline of existing DFlash speculators, these new draft models achieve an additional 5 - 20% speedup on a wide variety of workloads.

That's enough to run Qwen 3.5 122B-A10B at over 1000 tok/s at concurrency 1 on a B200 node. Here's roughly what that looks like, compared to the model running without any speculation at 250 tok/s, using the token timing simulator from our LLM Engineer's Almanac:

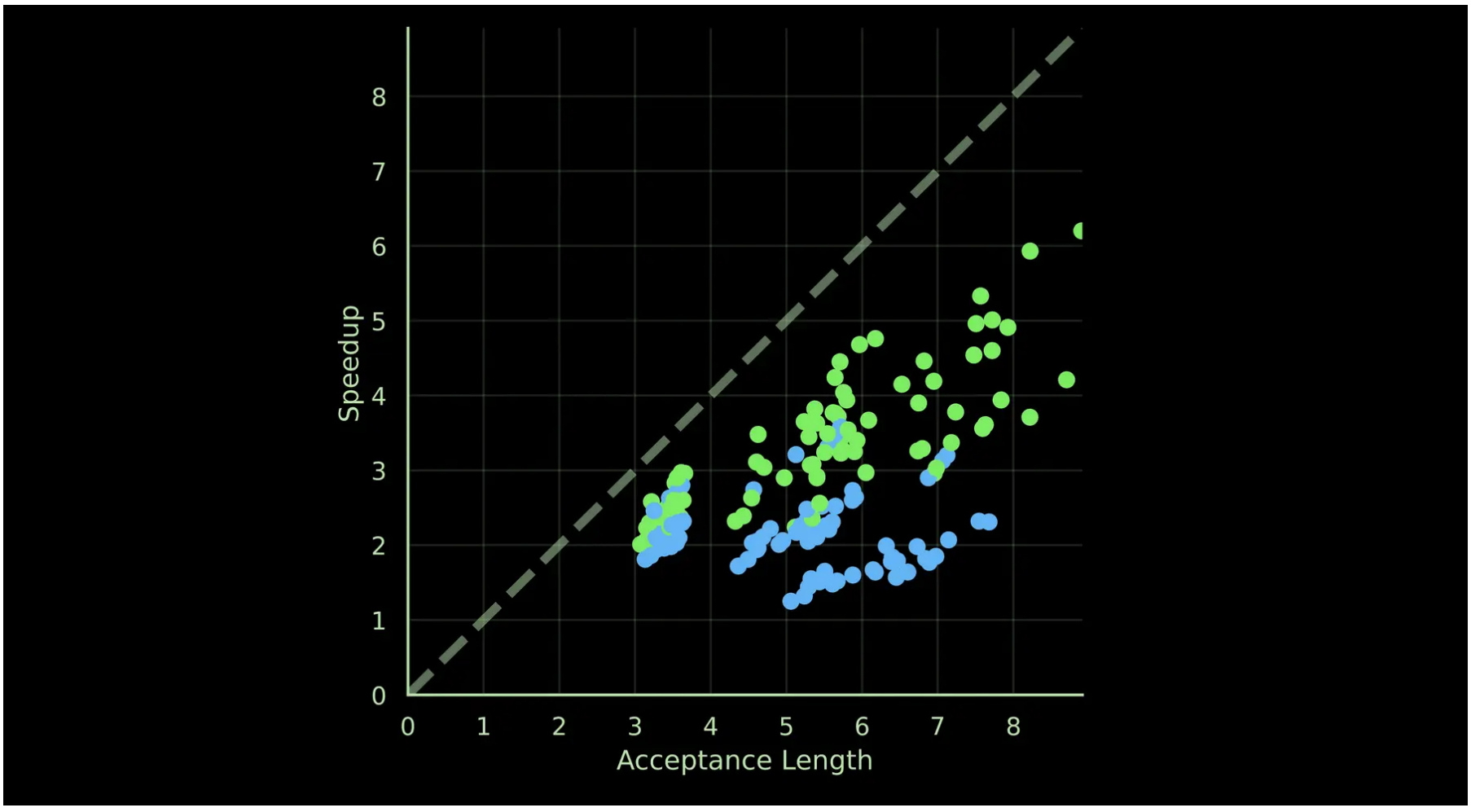
Furthermore, they better preserve their acceptance lengths on very long context tasks, like agentic software engineering.

Below, we explain why we're bullish on speculation for LLM inference acceleration — and as part of the whole continuous improvement cycle for AI applications. But first, **a tl;dr with the high-level takeaways**.

To first order, **speculative decoding is the only engine optimization that matters** for achieving state-of-the-art inference performance at high interactivity. Days of back-breaking kernel optimization work by expensive CUDA engineers or carefully profiling and lifting host-side bottlenecks delivers speedups measured in small percentage points. It is a grind and a game of inches. **Many inference providers wasted many engineering hours building proprietary engines filled with these optimizations**.

Speculative decoding delivers much larger speedups — measured in integral factors like 2x or 3x, not 2% or 3%.

The chart below shows speedups we've observed in speculators that we have trained, and in the built-in MTP baselines, as a function of speculator quality. You can explore the data in a Modal Notebook if you're interested in the details.



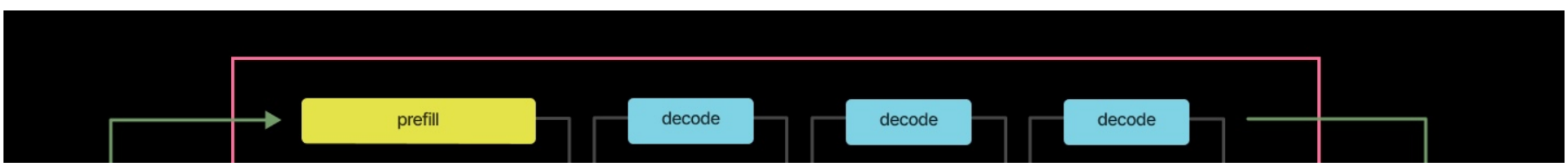
Proper support for speculative decoding is, therefore, more important than other optimizations. Open source inference engines like SGLang and vLLM have cottoned on and, in our experience, closed the gap with proprietary engines. Speculative decoding also generally composes with other work on inference engine performance.

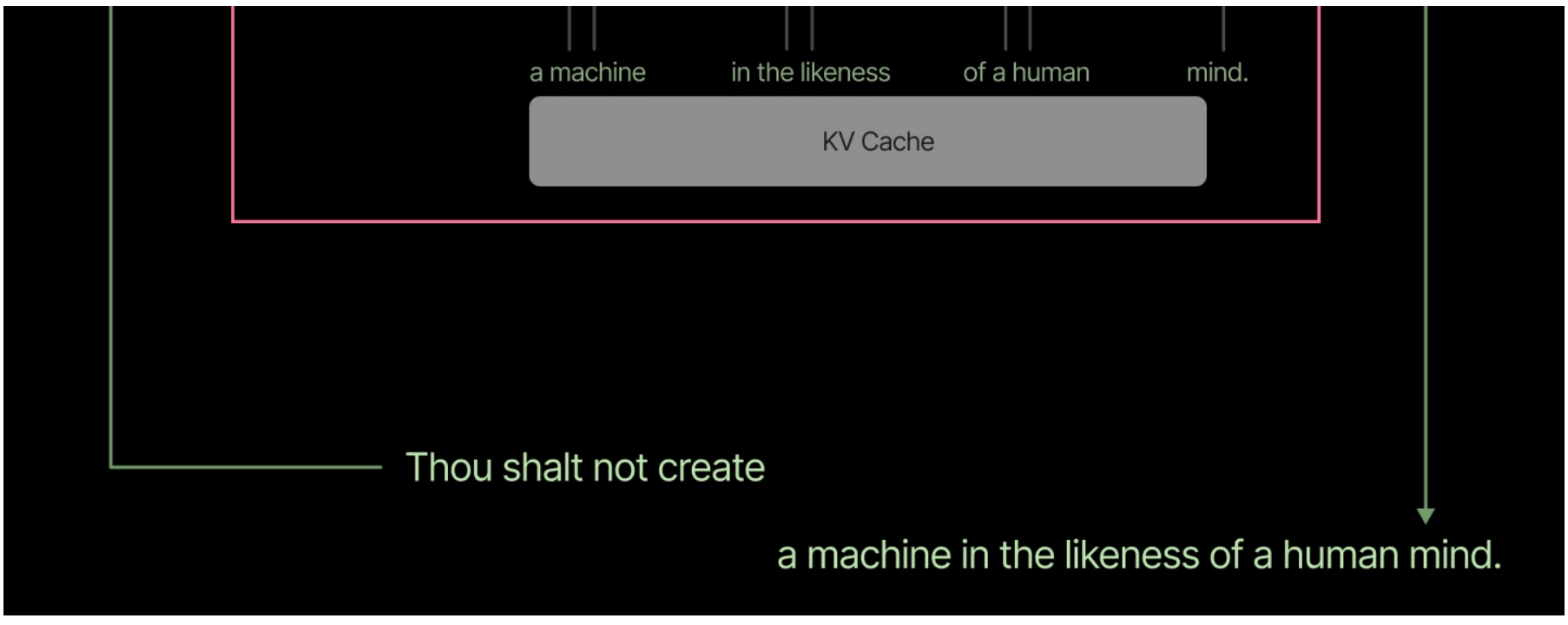
Finally, **when speculative decoding is customized to domain-specific data from an application, it delivers truly unbeatable speedups**. That means speculative decoding is Bitter Lesson-pilled: because speculative decoding relies on machine learning under the hood, the **speedup increases when you just throw more data and compute at the problem** — no cracked kernel engineers required. That means it can ride the same exponentials of continuous improvement in hardware, algorithms, autoresearch, and scale as the AI application it accelerates.

Speculative decoding is so critical to the success of contemporary self-hosted inference that you might even say that speculation Is All You Need.

What is speculative decoding and why is it so important?

A brief recap: speculative decoding (aka "spec dec") losslessly accelerates the "decode" phase of LLM inference, during which tokens are generated as output in response to an input.

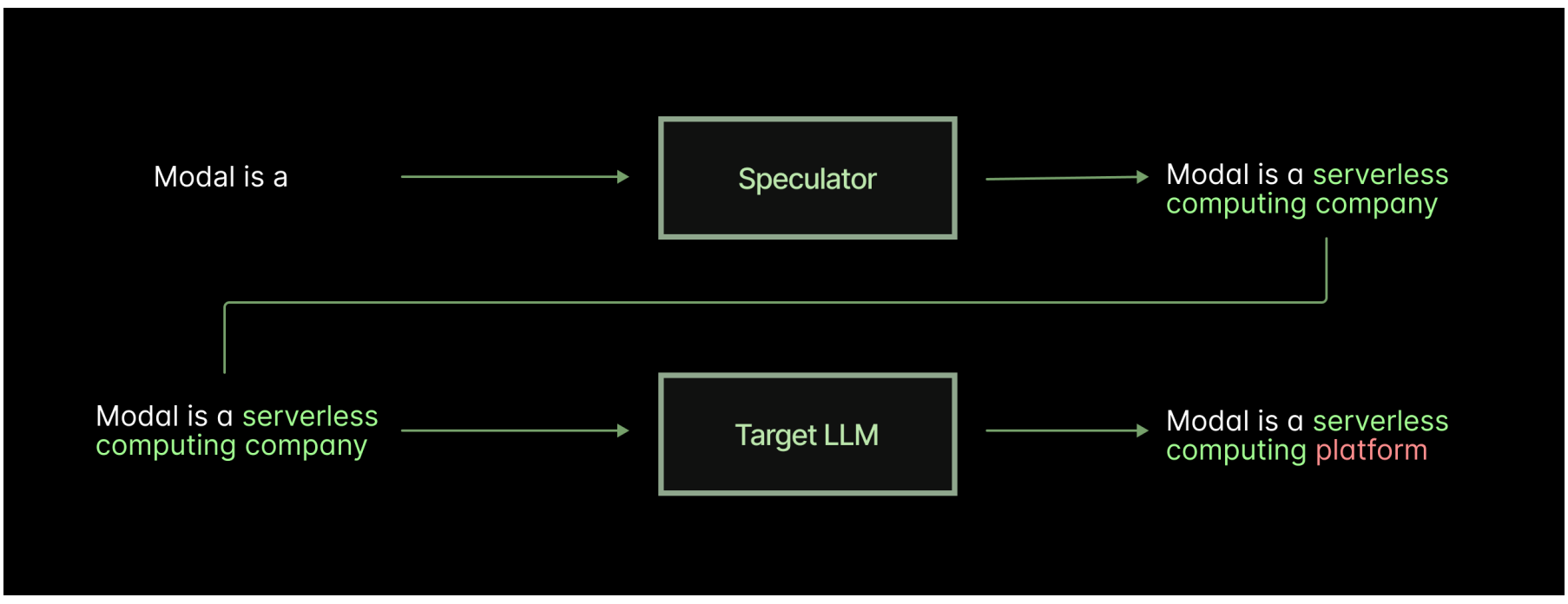




This is a serial operation, because Transformer (and Transformer-like) language models generate output tokens *autoregressively* — based on their own outputs.

Speculative decoding turns this serial work into parallel work by passing in a set of tokens generated by another system, the speculator (aka “drafter” aka “draft model”). These tokens can be processed in parallel by the target model, just as the model processes the input tokens in parallel (during the “prefill phase”).

The target model computes its own output probabilities for the tokens and applies a resampling technique (for the ‘heads, usually sequential rejection sampling). For deterministic/greedy decoding (aka temperature 0), this just means accepting the prefix of tokens that the target model would have output autoregressively, rejecting all tokens after that, and inserting a token predicted by the target.



To repeat, this acceleration is *lossless*. Speculative decoding produces sample sequences from the same distribution as the target model (up to non-determinism sources like floating point accumulation re-ordering).

The core intuition for speculative decoding is the same intuition as that for speculative execution in microprocessors: sequential execution is so expensive that parallel execution of work you might throw away is still worth it.

Each decode pass is quite like a sequential scan in a database. You load all active weights (many gigabytes)

from GPU memory into the GPU SMs each pass, just as you must load an entire table into the processor during each scan. Speculative decoding is then akin to eager construction of materialized views riding along with another sequential scan in databases. Work is wasted if no query ever reads the view, but you're generally I/O bandwidth-bound, so wasted work is "free", marginally.

The catch is that you need to create this speculator model. Early approaches either used classic ML (e.g. an n-gram model), which led to few tokens being accepted, or another neural network, which led to high drafting cost. Both reduce the speedup from speculation, as we more precisely model and simulate below. Contemporary architectures, like MTP, EAGLE-3, and DFlash use speculator models that piggy-back on the target model's past computations.

These speculator models are light-weight, result in high acceptance lengths, and are relatively easy to train.

But relatively is doing a lot of work here. Machine learning projects are notoriously hard to manage and prone to failure. Luckily, the hardest problems don't pertain to speculator training.

Training speculators is ML on easy mode.

In machine learning, we create a machine that mimics some data-generating process in the world. We call that machine an ML model — unless it's good enough at something humans get paid for, in which case we call it an "AI".

One of the trickiest parts of this setup is the "in the world" part. The world generates data promiscuously, but almost all of that information is lost. Only a sliver gets captured by computer systems, and the process of observing the world perturbs the data-generating process. There is also almost always a large gap between the data you can collect and the underlying process you truly want to model.

In speculator training, the gap is eliminated and the data is plentiful, because *the data-generating process is another ML model*! It's trivially easy to collect more data, reshape it, or generate it on the fly during training. No need to re-assign your software engineers to Macrodata Refinement. There's also a goal metric that's nigh-impervious to Goodharting: acceptance rate by the target model and speedup of the target inference system.

Infrastructure in ML is hard, but luckily we know infrastructure. We've been able to train speculators 40x faster by building our own speculator training framework that takes full advantage of Modal's fast autoscaling and massive concurrency. You can try out a similar Modal-native training framework designed for post-training reinforcement learning, here.

Training custom speculators is useful because the needle on acceptance lengths, and hence speedup, can be

meaningfully moved on datasets of the scale created by application usage — on the order of tens of thousands or hundreds of thousands of tokens. We have seen fine-tuned models increase acceptance lengths from a baseline of 3 to over 9. That's the difference between a 25% speedup and a 3x speedup, as we demonstrate below.

Intuition for the importance of acceptance length from three simple models

To see why improving acceptance length is so important, let's model speculative decoding three ways:

- Some light-weight simulations in a production inference engine, SGLang, via acceptance mocking
- A dead simple mathematical model for intuition
- A more sophisticated roofline model to flesh that intuition out

Each of these techniques allows us to understand, appreciate, and engineer for the speedups from high acceptance lengths without running a bunch of expensive, time-consuming ML training runs.

Mocking speculation in SGLang

Before we decide to commit resources to training a speculator, we'd like to understand what kinds of speedups we might observe from speculation.

With other methods to accelerate inference, like kernel optimization, we can readily mock the workload. For instance, in SGLang, you can pass `--load-format=dummy` to get random weights and send in random token IDs during benchmarking. This can have some small effects on behavior, especially with numerically unstable kernels. But it's close enough to production to accelerate a lot of optimization work.

Mocking decouples the algorithm development from data semantics. There are a number of benefits here. For instance, large terabyte-scale model weights or datasets don't need to be moved around from storage to development servers. They don't even need to be loaded from disk or pass through CPU RAM — they can be generated directly on device!

Mocking speculation is trickier. The speculator is its own ML model, so you can't just run it on random tensors, or the acceptance lengths will plummet. The speculator is operating out of distribution! That would seem to rule out dummy weights. And the input data is even trickier, since that needs to closely match what the actual system will see, especially for fine-tuned speculators. More on that next week!

SGLang includes a little-known environment variable that solves this problem: `SGLANG_SIMULATE_ACC_LEN`. Setting this flag mocks the acceptance behavior by just accepting generated tokens up to a certain length without regard to the target model's probability. A speculator and target model with random weights will still

produce roughly the same work and so take roughly the same time.

As with any perturbation of complex numerical code, there are of course ways that this causes divergence from real workloads. However, it gives us another way to check our models and to predict the benefit of a better-trained speculator.

If we benchmark Qwen 3.5 27B at concurrency 1 on a B200 with 4Ki tokens in, 4Ki tokens out random data and simulate acceptance lengths between 1 (autoregressive) and 8, we see substantial speedups.

Acceptance Length	**Output tok/s**	**Speedup**
1	75	1x
2	140	1.86x
4	268	3.57x
8	422	5.62x

We can add more lines of evidence and develop intuition for where the speedup comes from and how it can be increased through mathematical modeling, which we turn to next.

A toy model of speculation

Let’s start with the absolute simplest model we can come up with for speculative decoding. In this model, we’ll see that the speedup from speculation is equal to the acceptance length.

First, let’s define the `speedup`:

We can readily model the throughputs as a function of

- the acceptance length, the number of tokens produced by speculation (`acc_len`), and

- the draft length, the number of tokens speculated (`draft_len`).

We model autoregressive decoding as “predict one token `draft_len` times and get out `draft_len` tokens”. We model speculative decoding as “pass in `draft_len` tokens at once and get out `acc_len` tokens”.

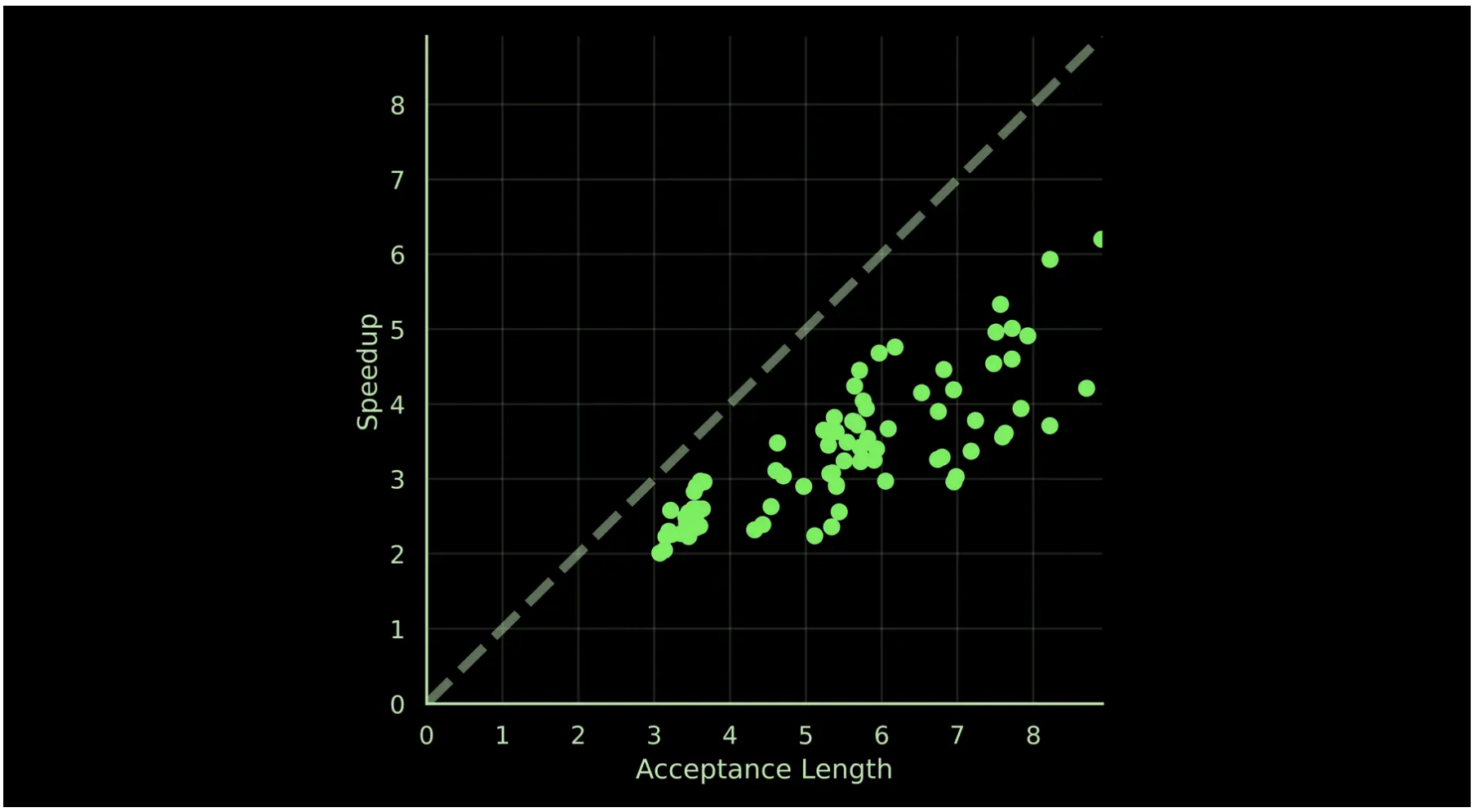
To start, let’s assume that model latency is the same for `1` token as it is for `draft_len` tokens. We plug those values in:

If we unpack this on our local whiteboard, we notice that many terms cancel — `draft_len / draft_len` and `model.latency(1) / model.latency(1)`. In fact, we just get:

This equal latency assumption isn’t always true, and it can be badly incorrect. Bear with us — we’ll justify when it

approximately holds and improve our model shortly.

But this simple model is surprisingly useful. For instance, below are the observed speedups for all the speculators we released this week, measured in the SGLang inference engine, with the very simple `speedup == acc_len` model plotted as a dashed line. Though the speedups are consistently overestimated, a linear trend is visible across the observed acceptance lengths.



So we have a useful rule-of-thumb for estimating speedups from acceptance length across practical values — linear, not quadratic or square-root or logarithmic. However, it can't actually be used to estimate speedups.

The overestimation occurs because we made a number of assumptions that are favorable to speculative decoding:

- The draft tokens add no latency to the target forward pass.
- The speculator forward pass adds no latency.

Let's fix these now.

Better modeling with rooflines

Note: the model in this section was developed using the model of speculation with DeepSeek-V4 Flash presented by Fergus Finn of Doubleword in this blog post. The numbers from that model were used as a reference for our implementation of optimal draft length calculation.

To incorporate the latency of the target forward pass as a function of load (including draft tokens), we build a simple model of that forward pass based on some simplifying assumptions:

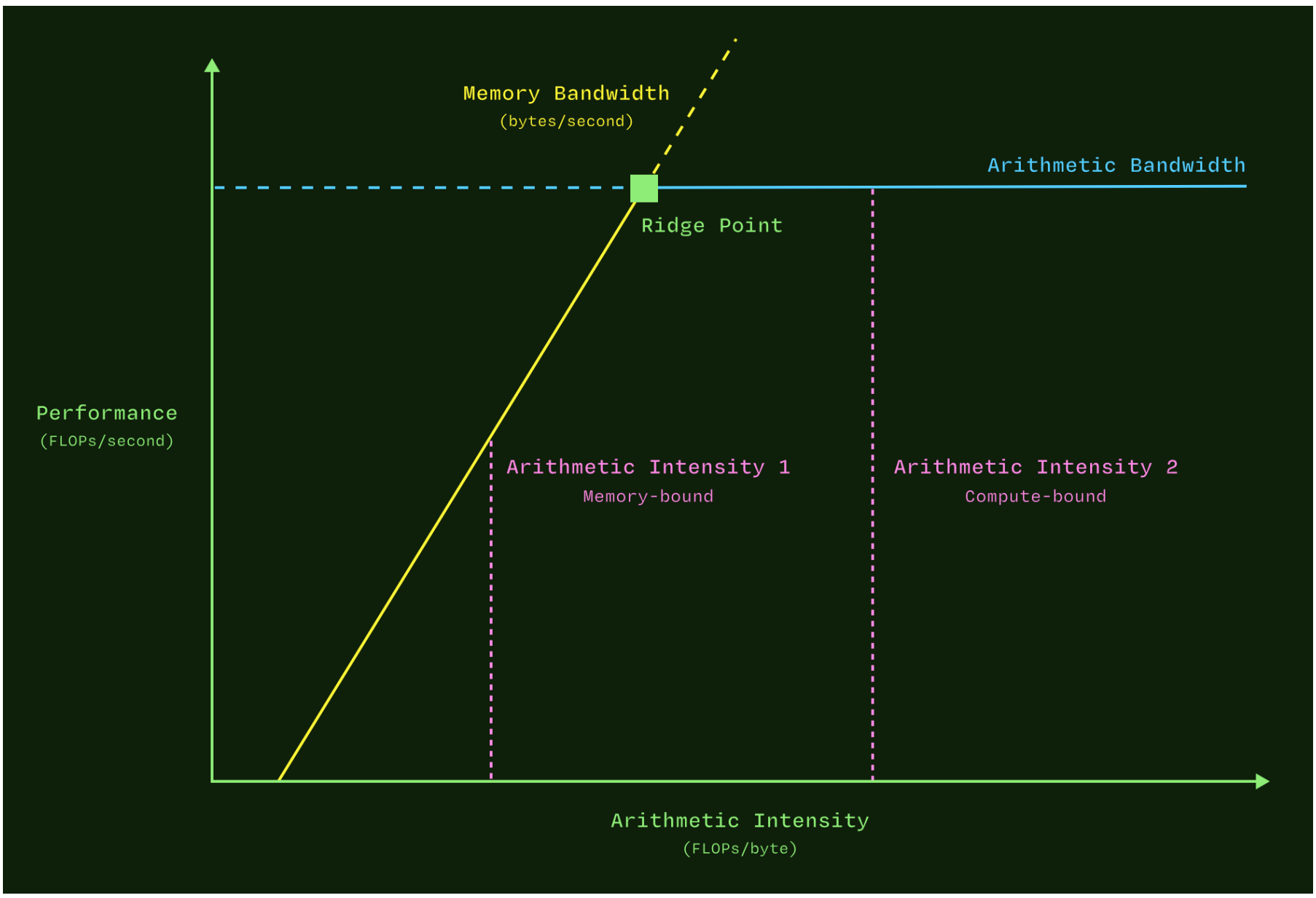
1. the memory bandwidth and arithmetic bandwidth of the accelerator is always fully utilized,

2. memory transfers and computation can be perfectly overlapped and all latency hidden, and

3. the host feeds work to the accelerator faster than the accelerator can complete it, avoiding overhead.

   These are all more accurate for larger models, longer sequence lengths, and larger batch sizes.

   With those assumptions, we can estimate the latency of the model based on how many bytes it needs to read, how many floating point operations (flops) it needs to do, and how long those take when exercising the full memory and arithmetic bandwidths. We simply take the higher of the two latencies (the compute lower bound or the memory lower bound). This is a roofline model of performance.



Mechanically, this calculation is easiest to do layerwise, since attention and matrix multiplication look so different. We ignore the other layers, since they are generally either very fast or can be overlapped with the longer-running operations via epilogues or other kernel fusions.

That means this modeling code looks something like:

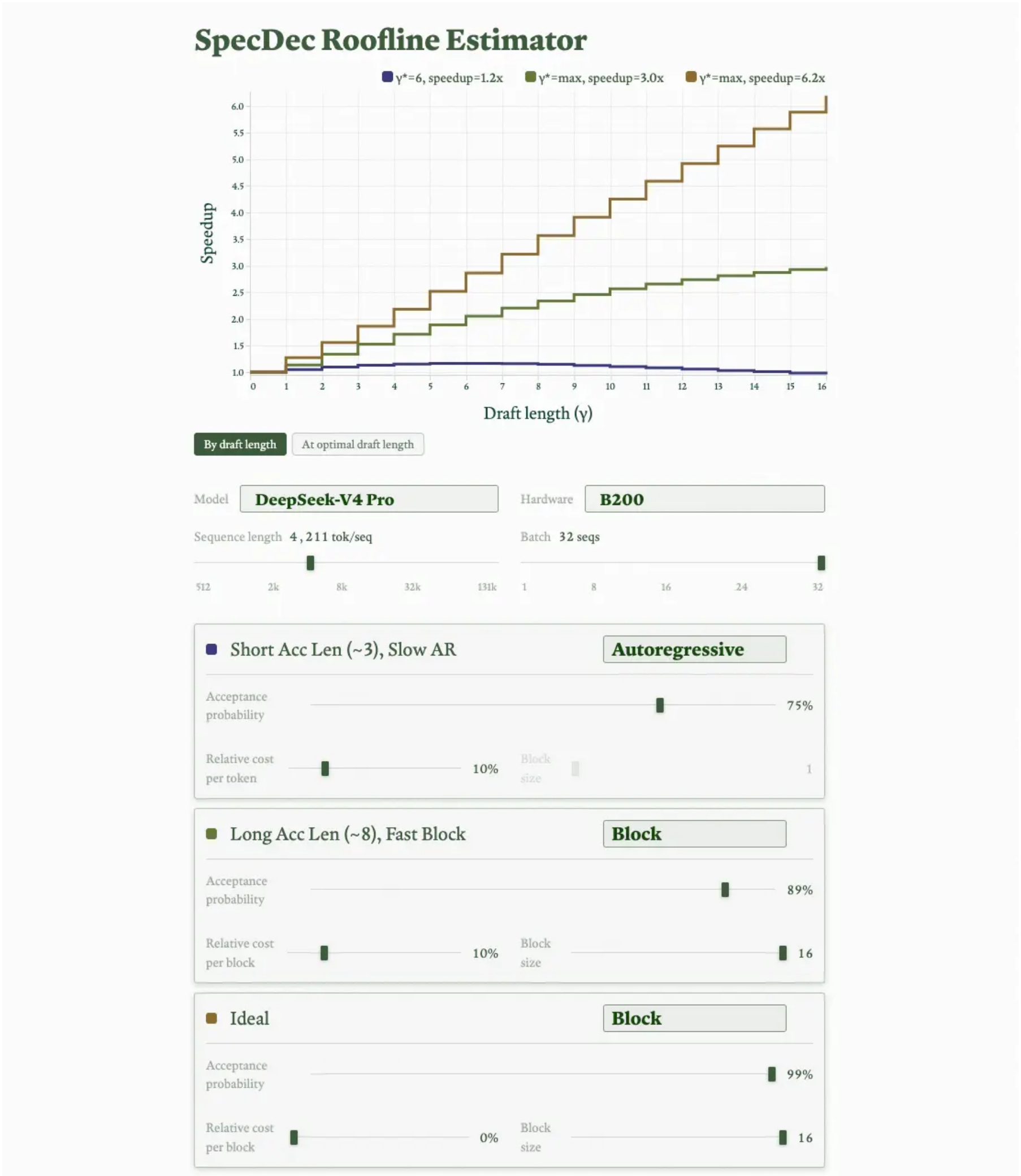
Note that the loaded bytes for attention and dense MLP modules are roughly constant over `query_tokens_per_seq` for small numbers of query tokens. This justifies the core approximation in our simple model — so long as the bound from compute latency isn't higher than the bound from memory latency, the latency we calculate in this model is approximately the same for a single query token and for several.

The relevant architectural data can be read off of a Hugging Face config and combined with some simple logic to calculate the flops performed and bytes read.

We choose to model the speculator's forward pass latency as a fixed percentage of the target model — something like 5% to 20% seems common. For drafters that are autoregressive, this is paid once per draft token. For drafters that produce blocks, like DFlash, this is paid once per block.

These calculations are pretty quick, so we can do them dynamically — in a browser, with JavaScript, no GPUs required. You can try that out here.

We walk through a sample output from this model below.



The chart compares the decode speedup from speculation, over the autoregressive baseline, for DeepSeek-V4 Pro deployed on a single B200 node processing a specific workload. The workload in this case is short

sequence length (~4k tokens/sequence) and high concurrency (batches of 32 sequences). The chart at the top shows the predicted speedup factor at a variety of different draft lengths for several drafters, along with the highest speedup factor. That speedup is achieved at the optimal draft length according to the model (or 16, whichever is lower).

Three drafters are compared. In gold is the “Ideal” drafter. This drafter has nearly perfect acceptance rate and completes its forward pass instantly. This provides a ceiling for speculation with this workload and hardware, at least according to roofline concerns. The drafter in blue represents a typical autoregressive MTP drafter, with a short acceptance length. The drafter in green represents a typical well-trained block drafter (like DFlash), which achieves a much higher acceptance length.

Using a faster drafter and improving acceptance length from ~3 to ~8 changes the predicted achievable speedup substantially. It goes from 20%, which is simply respectable, to 3x, which is enough to open new applications or markets. Further improving acceptance lengths would allow for even larger block sizes.

This simulator is imperfect — because of the limitations of the roofline model, as described above, and because it doesn’t attempt to model communication of tensors across GPUs. However, we have found that it is more correct than the simple toy model. For instance, it correctly predicts that speculator speedup is non-monotonic (”u-shaped”) as a function of batch size for mixture-of-experts models, like DeepSeek-V4 Pro, but monotonic for dense models, like Qwen 3.5 27B. You can see that here by first viewing the default version of the chart, then selecting Qwen 3.5 27B.

What’s next?

In this article, we’ve considered the current production state-of-the-art for open source speculative decoding. What does the future look like?

We anticipate the following trends:

- adaptive speculation
- improvements to speculator architectures
- improvements to speculator implementations
- embrace of lossy speculative decoding
- iterative speculator training and distillation

Adaptive speculator training

First, we think it involves more custom speculators. Production data experiences drift. User behavior changes over time, and inference systems, including speculators, need to adapt. There's excellent prior work on this from Together. We are experimenting with adaptive speculator training and look forward to deploying it to and with our users.

But adaptive speculation isn't as simple as "check acceptance length on a cron". Some changes in user behavior are seasonal, rather than persistent. For instance, one company building adaptive speculation on Modal has two distinct bases on opposite sides of the globe that each speak their own (mix of) languages. That limits the speedup of a fixed-capacity speculator. Worse still, a poorly-implemented adaptive speculator system was triggering retrains every time the user base shifted, twice per day, when it could just maintain two "regional speculators" it retrains much less frequently.

Better speculator architectures

Second, we think it involves more work on speculator architectures. DFlash introduced two new ideas we like (as we explained in detail on the LMSys Org blog). The KV injection technique allows for deeper, smarter drafters. The single-step diffusion/"BERT if you squint" approach of DFlash is also better for high-arithmetic-intensity hardware.

But the hot new trick in diffusion models these days is using flow maps. Diffusion models induce a vector field via their denoising operation, and typical inference approaches follow that field step by step. Flow maps take in initial states and step counts and output the final state — like a lookup table of integrals, rather than a sequential integrator. With some clever math, you can write down objectives to concurrently train a multi-step diffusion model and an integrator for all "jumps" of multiple steps, including the single-step denoiser. For speculation, that gives you an additional knob for trading off acceptance length and drafter latency, without needing to retrain the drafter.

Better speculator implementations

Third, we think that the implementations of speculators will improve. When integrating DFlash with SGLang, we found that we needed to write a fused kernel for the KV injection step to improve utilization and throughput.

There's a generic lesson here: speculators must run faster than the target, which generally means they are smaller. Smaller models struggle to saturate the memory and arithmetic bandwidth and more readily incur host overhead when running on the same hardware that is optimal for larger models. Disaggregation of speculator and target is intriguing, but we expect communication latency to relegate it to a minority of niche use cases.

The limit of kernel fusion is a megakernel — a single kernel that runs an entire model. These have previously

been rare due to the engineering cost of megakernel authoring, which makes normal kernel authoring look like bash scripting. Contemporary approaches to megakernels, like this work from Hazy Research, use an on-device "interpreter" to run instructions from a higher-level program at maximum concurrency and with maximum reuse, with impressive results. And engineering costs are going down with the rise of coding agents, including in kernel authoring. See this blog post from our customer Recursive Superintelligence for intriguing early results (and cautionary tales about reward hacking).

Lossy speculative decoding

Because speculative decoding is lossless, it's an excellent option for generic inference providers who are offering an API that returns samples from a particular model. But this losslessness guarantee is constraining.

In light of recent events in the world of proprietary inference, more organizations are considering the option of owning their inference and using open models. When you own your inference, you can optimize it for your own requirements — including trading some change in model behavior for a massive improvement to latency.

A vision of a flywheel

Let's sketch out how speculators and in-house inference work together to deliver a system that continually improves in quality, latency, and cost.

The typical organization prototypes inference applications with a generic foundation model, often via a proprietary provider.

Once the application is prototyped, you can devote the engineering effort to host inference. At Modal, we're making that as easy as possible — more on that next week. The traces and evals developed during prototyping make that transition seamless. Generic pre-trained speculator models ensure that costs and latency are controlled, with no cost to quality.

Once you have a few tens of thousands of samples from this self-hosted inference, you can then train a custom speculator. The custom speculator can be used to reduce costs at fixed latency or reduce latency at fixed costs.

Once you have even more samples, you can distill (hard or soft) the deployed target model into a smaller one, further reducing either or both of latency and cost without harming quality. Critically, the same data sources you are using to build evals and train speculators can be used for distillation.

And now you repeat — with the smaller model as your new baseline.

Since this entire stack is based on compute and data, it is all accelerated by improvements in algorithms, models, and hardware. That means the compounding loop of quality, cost reduction, and performance from

distillation and speculation is riding on top of many compounding loops of improvements inside and outside your organization.

If you're interested in building this, reach out. If you're interested in building the infrastructure that supports this, we're hiring.